

MusicOS: A Permissionless Protocol for Programmable Music Rights and Distribution

Version 1.0 — February 2026

Unconfirmed Labs
os@unconfirmed.com

Abstract. The music industry operates on infrastructure built for the physical distribution era: opaque royalty pipelines, fragmented ownership databases, and intermediaries whose value increasingly derives from positional control rather than creative contribution. An estimated \$2.5 billion in royalties go unclaimed annually due to incomplete or conflicting metadata, while artists remain structurally unable to verify how revenue flows from consumption to payment. The emergence of generative AI compounds the problem by commoditizing audio production itself, eroding the already-thin margin artists capture from distribution.

We present MusicOS, a permissionless protocol on the Sui blockchain that provides a comprehensive on-chain data model for music compositions, recordings, releases, and the parties involved in their creation. MusicOS introduces *programmable music* and embraces the ideals of *open source music*: each composition, recording, and release is a canonical, addressable object on a public ledger, capable of evolving over time, accumulating media and metadata, and exposing its full state to any consumer. Recordings include not just final masters but individual stems (the source code of a recording), making the creative building blocks of music openly available and programmable. By encoding the full structure of music rights (the distinction between compositions and recordings, multi-party credit attribution, basis-point revenue splits, and share-token ownership) directly into smart contracts, MusicOS creates a verifiable, deterministic, and agent-compatible distribution layer where the economics of openness exceed the economics of control.

Table of Contents

1	Introduction	3
2	Background and Motivation	3
2.1	The Structure of Music Rights	3
2.2	Infrastructure Failures in the Current Industry	4
2.3	The Commoditization of Audio	4
2.4	Open Source Music	5
2.5	Limitations of Existing Blockchain Approaches	5
3	Protocol Design	6
3.1	Design Principles	6
3.2	Core Objects	6
3.3	Object Lifecycle and State Machines	7
3.4	The Party and Credit System	7
3.5	Audio Ingestion	8
3.6	Deterministic Object Addressing	8
4	The Extension System	8
4.1	Motivation: The WordPress Model	8
4.2	Registration and Witness-Gated Access	9
4.3	Raw UID Access and Future Compatibility	9
4.4	Extension Isolation and Trust Model	9
5	Revenue Distribution and Token Economics	10
5.1	Share Tokens	10
5.2	Basis-Point Revenue Splits	10
5.3	Release Revenue Distribution	10
5.4	Reward Pools	11
5.5	Multi-Currency Support	11
6	The Deal Protocol	11
7	Decentralized Media Storage	11
8	Agent Compatibility and Programmable Music	12
8.1	From DDEX Messaging to Declarative Execution	12
8.2	Agent-Native Interfaces	12
8.3	Programmable Music as a New Distribution Format	12
9	Comparison with Related Work	13
10	Discussion	14
10.1	Trust Assumptions	14
10.2	Protocol Neutrality	14
10.3	Why Sui	14
10.4	On-Chain Cost Analysis	15
10.5	Limitations and Trade-offs	15
11	Conclusion	16

1 Introduction

The global recorded music industry generated \$28.6 billion in revenue in 2023¹, yet the infrastructure connecting creators to that revenue remains remarkably primitive. Ownership data is scattered across disconnected databases maintained by performing rights organizations (PROs), mechanical licensing bodies, publishers, labels, and distributors, each operating with incomplete and often contradictory records. When a song is streamed, the revenue it generates passes through a chain of intermediaries, each applying proprietary calculations that the rights holders themselves cannot independently verify. The result is an industry where the people who create music are structurally the least informed about how it is monetized.

This paper introduces MusicOS, a permissionless protocol built on the Sui blockchain that provides the foundational data model and economic logic for music rights management and distribution. MusicOS does not aim to replace the music industry’s participants (artists, labels, publishers, distributors, and platforms all have meaningful roles) but rather to replace the *infrastructure* those participants operate on. The distinction is important: MusicOS is not an application but a protocol, not a platform but a substrate.

The protocol makes three primary contributions:

1. **A faithful on-chain data model for music rights.** MusicOS encodes the actual structure of music industry rights (the legal distinction between compositions (written works) and recordings (audio performances), multi-party credit attribution with role-specific classifications, and hierarchical revenue splits) rather than simplifying it into token-gated access or NFT collectibles.
2. **A permissionless extension architecture.** Inspired by WordPress, MusicOS allows any developer to deploy extension packages that attach new functionality to core protocol objects without requiring modification to or permission from the core protocol. Extensions interact with parent objects through witness-gated UID access, enabling immediate adoption of new Sui framework primitives without core protocol upgrades.

3. **Programmable music as a distribution format.** By binding audio files stored on decentralized storage (Walrus) to on-chain economic logic (ownership shares, revenue splits, credit attribution, and extension-defined behavior), MusicOS creates a new kind of distributable artifact: one where the music and its economics are inseparable and machine-readable.

The remainder of this paper is organized as follows. [Section 2](#) provides background on the structure of music rights and the infrastructure failures that motivate MusicOS. [Section 3](#) presents the core protocol design. [Section 4](#) details the extension system. [Section 5](#) describes the revenue distribution model and token economics. [Section 6](#) covers the deal protocol for recording-to-release authorization. [Section 7](#) discusses decentralized media storage via Walrus. [Section 8](#) introduces the concept of agent compatibility and programmable music. [Section 9](#) compares MusicOS with related work. [Section 10](#) discusses trust assumptions and limitations.

2 Background and Motivation

2.1 The Structure of Music Rights

To understand why music industry infrastructure is broken, one must first understand how music rights actually work. Every piece of recorded music involves at least two distinct sets of rights:

Composition rights cover the underlying written work (the melody, harmony, and lyrics). These are held by songwriters and publishers. A single composition can have multiple recordings (a cover version, a remix, a live performance), and each recording generates revenue that must be split between the composition rights holders and the recording rights holders.

Recording rights (often called “master rights”) cover the specific audio performance of a composition. These are held by performing artists and labels. A recording cannot exist without an underlying composition, and the revenue it generates is subject to a pre-negotiated split between the composition and recording sides.

Releases (albums, EPs, singles) aggregate multiple recordings into distributable products. Each recording on a release may have different rights holders, different

¹IFPI. *Global Music Report 2024*. International Federation of the Phonographic Industry, 2024.

composition–recording splits, and different revenue allocations.

This three-layer structure (composition, recording, release) is not an implementation detail. It is the legal and economic reality of the music industry, codified in copyright law across virtually every jurisdiction. Any protocol that claims to represent music rights must model this structure faithfully. Flattening it into “one NFT per song” discards the information that the industry’s economic relationships depend on.

2.2 Infrastructure Failures in the Current Industry

The music industry’s infrastructure failures are interconnected and self-reinforcing:

The black box problem. When a listener streams a song on Spotify, Apple Music, or any other digital service provider (DSP), the revenue generated by that stream enters a pipeline that the rights holders cannot independently audit. DSPs report usage to distributors, who report to labels, who report to artists, each step introducing latency, opacity, and the potential for error or extraction. An artist receiving a quarterly royalty statement has no mechanism to verify its accuracy against the underlying consumption data.

The unmatched royalties crisis. The Music Modernization Act of 2018 established the Mechanical Licensing Collective (MLC) in the United States to address the problem of unmatched mechanical royalties (payments owed to songwriters and publishers that cannot be distributed because the ownership data is incomplete or conflicting). As of recent reporting, the MLC holds hundreds of millions of dollars in unmatched royalties.² The crisis is a predictable consequence of an industry that has never had a single, authoritative, openly verifiable source of ownership data.

Licensing complexity as a barrier. The distinction between composition rights and recording rights means that any entity wishing to use a piece of music commercially must often negotiate with multiple rights holders across multiple organizations. PROs (ASCAP, BMI, SESAC) handle public performance rights for compositions. The MLC handles mechanical rights. Labels handle master use licenses. Publishers handle synchronization rights. Each operates its own database, its own

payment schedule, and its own fee structure. The result is a system where licensing a single song for a single use case can require navigating half a dozen organizations with no shared infrastructure.

Data fragmentation. There is no canonical global database of music ownership. The International Standard Recording Code (ISRC) identifies recordings, and the International Standard Work Code (ISWC) identifies compositions, but these identifiers are inconsistently applied, frequently duplicated, and not programmatically linked to ownership or payment information. The meta-data that connects a stream to a payment is maintained in proprietary databases that do not interoperate.

MusicOS addresses the bridging problem without polluting core objects with legacy identifiers. SuiNS,³ Sui’s on-chain name service, provides a natural resolution layer: an ISWC code such as T-123.456.789-Z can be expressed as `t123456789z.iswc.sui` and pointed to the corresponding Composition object. Similarly, ISRCs can resolve to Recordings, and any other industry code (IPI, ISNI, etc.) can be mapped through its own SuiNS subdomain. Operators can run verified name services that resolve legacy codes to their on-chain counterparts, providing interoperability with existing industry infrastructure without requiring changes to the core protocol objects or reliance on off-chain indexing or a custom on-chain registry.

2.3 The Commoditization of Audio

These infrastructure failures exist against a backdrop of accelerating change. Generative AI has reduced the marginal cost of audio production toward zero. Tools for AI-assisted composition, arrangement, and mastering are widely available and improving rapidly. The implications for artists are stark: if the primary unit of distribution is the audio file, and audio files are becoming trivially cheap to produce, then the economic position of human creators within the distribution pipeline will continue to erode.

The response is to expand what music distribution means. When audio alone is a commodity, the value that human artists contribute (creative intent, cultural context, collaborative relationships, the social and economic networks surrounding a work) must be captured in the distribution format itself. An audio file on a server cap-

²U.S. Copyright Office. *Mechanical Licensing Collective: Annual Report*. Library of Congress, 2023.

³SuiNS. *Sui Name Service*. <https://suins.io>.

tures none of this. A programmable on-chain object that binds the audio to its ownership structure, credit attribution, revenue logic, and extensible metadata captures all of it.

MusicOS proposes that the unit of music distribution should not be the audio file. It should be the *programmable composition-recording-release structure*: a machine-readable, economically complete representation of a musical work and all the relationships it embodies.

2.4 Open Source Music

Open source software runs the world. The operating systems, web servers, databases, programming languages, and AI frameworks that underpin the global digital economy are overwhelmingly open source. The logic is well understood: when source code is transparent, inspectable, and composable, it enables a combinatorial explosion of derivative innovation that closed systems cannot match.

MusicOS applies this logic to music. The protocol requires that recordings include not just the final master audio but also the individual stems (the isolated vocal, instrumental, and production layers that constitute the *source code* of a recording). Traditional streaming platforms distribute only the “compiled binary”: a mixed-down audio file that consumers can listen to but cannot meaningfully inspect, decompose, or build upon. MusicOS distributes the source.

To anyone familiar with the music industry, this is a radical proposition. Stems are the thing you *never* share. The conventional wisdom, drilled into artists by managers, labels, and hard-won experience, is to guard your stems, your sessions, your raw materials, because every relationship in the traditional industry is fundamentally adversarial. Labels recoup before artists earn. Publishers collect before songwriters see a statement. Distributors extract fees at every handoff. In this environment, sharing the building blocks of your work is irrational: it gives away leverage to parties whose economic interests are structurally misaligned with yours.

MusicOS changes the calculus. By replacing opaque intermediaries with deterministic on-chain logic (where revenue splits are encoded, not negotiated behind closed doors; where ownership is verified, not asserted by whoever holds the contract), the protocol removes the adversarial conditions that make stem hoarding rational. When artists can verify exactly how every dollar flows,

when share tokens give them direct economic participation rather than a royalty statement they cannot audit, and when extensions can unlock new revenue streams from the very stems they contribute, the ROI of openness exceeds the ROI of control.

This is the deliberate trade at the heart of MusicOS: in exchange for open-sourcing their music, artists gain access to an on-chain economy where their work is fully programmable. When stems are on-chain and addressable, the range of products that can be built on top of a musical work expands dramatically. Remix tools, AI-assisted production workflows, interactive listening experiences, stem-level licensing, educational applications, and entirely new creative formats become possible, not through ad hoc partnerships with platforms, but as permissionless extensions that anyone can build. The stems are the API surface of the recording.

Open source software demonstrated that transparency and composability create more economic value than they capture. Open source music is the same proposition: by making the creative building blocks of music openly available and programmable, MusicOS enables an ecosystem of derivative value that benefits creators, developers, and listeners in ways that closed distribution cannot. The name itself encodes this dual identity: MusicOS is both an operating system for music (the foundational layer on which applications are built) and a commitment to open source as the distribution model.

2.5 Limitations of Existing Blockchain Approaches

Several blockchain-based music projects have emerged in recent years, each addressing some aspect of the problems described above. Audius provides a decentralized streaming platform. Sound.xyz enables artists to sell limited-edition recordings as NFTs. Royal allows fans to purchase fractional streaming royalty rights. These projects have demonstrated demand for blockchain-based music infrastructure, but they share common limitations:

Platform, not protocol. Most blockchain music projects are applications with their own user interfaces, token economies, and governance structures. They are destinations, not infrastructure. An artist using Audius is using Audius; they are not building on a permissionless protocol that any application can integrate.

Simplified data models. Existing projects typically model music as a single asset (an NFT, a token, a

stream) rather than faithfully representing the composition-recording-release hierarchy that governs real-world music rights. This simplification may be adequate for collectibles and fan engagement, but it cannot support the full range of music industry operations: mechanical licensing, synchronization rights, multi-party revenue splits, and inter-organizational data exchange.

Permissioned extension. Adding new functionality to existing blockchain music platforms typically requires the platform’s cooperation. There is no mechanism for independent developers to extend the protocol’s capabilities without modifying the core codebase or obtaining approval from its maintainers.

MusicOS addresses each of these limitations. It is a protocol, not a platform. It models the full structure of music rights. And its extension system allows anyone to build on top of it without permission.

3 Protocol Design

3.1 Design Principles

MusicOS is built on the Sui blockchain using the Move programming language.⁴ The protocol’s design is guided by four principles:

Fidelity to real-world rights structures. The on-chain data model must faithfully represent the legal and economic relationships that govern music rights. Simplifications that discard information needed for real-world operations are rejected in favor of completeness.

Immutability after publication. Core music metadata (compositions, recordings, and releases) follows a state machine from mutable (Initialized) to immutable (Published). Once published, an object becomes a shared object on Sui, readable by anyone and modifiable by no one. This ensures that the authoritative record of a musical work cannot be retroactively altered.

Capability-based authorization. Every mutable operation on a MusicOS object requires a corresponding admin capability (CompositionAdminCap, RecordingAdminCap, ReleaseAdminCap, PartyAdminCap). Capabilities are standard Sui objects that can be transferred, shared, or held in custody, enabling flexible governance models

without hard-coding authorization logic into the protocol.

Extensibility without permission. The core protocol provides the data model and lifecycle management. All additional functionality (revenue distribution, reward pools, metadata enrichment, licensing automation) is implemented as extensions that any developer can deploy independently.

3.2 Core Objects

MusicOS defines four primary on-chain objects:

Party. Represents any entity involved in the creation or management of music: an individual artist, producer, engineer, or a group (band, orchestra, choir). Groups can contain references to individual party members. Parties are the foundational identity object in MusicOS: compositions, recordings, and releases all reference parties in their credit maps.

```
Party {
  id: UID,
  kind: Individual | Group(VecSet<ID>),
  name: String,           // max 200 bytes,
  max 200 members
}
```

Composition<S>. Represents the underlying written work (melody, lyrics, harmony). Parameterized by a phantom share token type `S` that links the composition to its ownership token. Contains the title, alternate titles, credits (a mapping of party IDs to roles), a basis-point revenue split, and optional content references (lyrics, chart, score, demo audio) stored on Walrus.

```
Composition<S> {
  id: UID,
  state: Initialized |
  Published(timestamp),
  title: String,
  alternate_titles:
  vector<String>,        // max 5
  credits: VecMap<ID>,
  Credit<Role>>,         // max 50
  split_bps: BPS,
  lyrics: Option<WalrusData>,
  chart: Option<WalrusData>,
  score: Option<WalrusData>,
```

⁴Sui Foundation. *The Move Programming Language*. 2024.

```
demo: Option<Audio>,
}
```

Recording<S>. Represents an audio performance of a composition. Contains rich musical metadata (genre, key, tempo, time signature), the master audio file, individual audio stems (minimum 2), cover art, and credit attribution with 23 role types. Each recording references its parent composition and captures the composition's revenue split at creation time.

```
Recording<S> {
  id: UID,
  state: Initialized |
  Published(timestamp),
  title: String,
  composition_id: ID,
  composition_split_bps: BPS,
  primary_genre_id: ID,
  secondary_genre_ids: VecSet<ID>, //
  max 3
  primary_artist_ids: VecSet<ID>, //
  max 20
  featured_artist_ids: VecSet<ID>, //
  max 50
  credits: VecMap<ID, Credit<Role>>, //
  max 150
  language: Option<LanguageCode>,
  is_explicit: bool,
  is_instrumental: bool,
  lyrics: Option<WalrusData>,
  musical_key: Option<MusicalKey>,
  time_signature: Option<TimeSignature>,
  tempo_bpm: Option<u16>,
  master: Audio, //
  required
  stems: vector<Stem>, //
  min 2, max 100
  cover_art: CoverArt,
}
```

Release. Represents a distributable collection of recordings (an album, EP, or single). Organizes tracks into discs, supports release-level credits (Primary and Featured artists), and contains the revenue distribution logic that splits incoming payments across tracks and their underlying compositions and recordings.

```
Release {
  id: UID,
  kind: Album | EP | Single,
  state: Initialized(bool) |
```

```
Published(u64),
  title: String,
  subtitle: Option<String>,
  description: String,
  credits: VecMap<ID, Credit<Role>>, //
  max 50
  discs: vector<Disc>, //
  max 20
  cover_art: CoverArt,
}
```

3.3 Object Lifecycle and State Machines

Compositions, recordings, and releases follow a two-state lifecycle:

Initialized — Published

In the **Initialized** state, the object is owned by its creator and can be freely modified: adding credits, setting metadata, attaching content. All mutating functions enforce state guards and abort if called on a published object.

The `publish()` transition is irreversible. It validates completeness constraints (compositions must have at least one credit and one content attachment; recordings must have credits, a primary artist, and at least two stems; releases must have a primary credit and valid track assignments), records a publication timestamp from the Sui Clock, and converts the object into a Sui shared object, publicly readable and permanently immutable.

This lifecycle ensures that published music metadata constitutes a permanent, tamper-proof record. Extensions can attach additional data to published objects via dynamic fields, but the core metadata is fixed.

3.4 The Party and Credit System

Attribution in the music industry is not a simple list of names. A recording typically involves dozens of credited parties, each with specific roles: producer, vocalist, mixing engineer, mastering engineer, instrumentalist, and many others. MusicOS models this with a generic credit system:

```
Credit<Role> {
  roles: vector<Role>,
}
```

The `Role` type parameter is specialized per context:

- **CompositionPartyRole:** Composer, Lyricist, Songwriter, Arranger, Adapter, Translator
- **RecordingPartyRole:** 23 role variants including Producer, Vocalist, MixingEngineer, MasteringEngineer, Instrumentalist, Conductor, and others. Many roles support optional level qualifiers (Lead, Featured, Assistant, etc.)
- **ReleasePartyRole:** Primary, Featured

A party must be credited on a recording before they can be designated as a primary or featured artist. Stem contributors must be credited on the recording before their contributions can be registered. These referential integrity constraints ensure that the credit graph is internally consistent.

3.5 Audio Ingestion

Audio in MusicOS is created through *witness-gated ingestion*. The `audio::new()` function requires a generic witness type parameter `Ingester` with the `drop` ability. Because only the `Move` module that defines a type can construct it, every `Audio` struct carries an on-chain attestation identifying exactly which ingester package created it. MusicOS is permissionless (anyone can build and deploy an ingester), but the witness type permanently stamps each `Audio` with its provenance, enabling downstream consumers to make informed trust decisions.

```
public fun new<Ingester: drop>(
  channels, bit_depth, sample_rate_hz,
  samples, data, _ingester: Ingester,
) -> Audio
```

The ingester's witness type is recorded in the resulting `Audio` struct as a `TypeName`, creating a permanent on-chain record of which ingester attested the audio data and its technical parameters (channels, bit depth, sample rate, sample count).

The purpose of ingestion is to attest that the audio file addressed by a given Walrus blob ID corresponds to the claimed audio metadata: that the file at that blob ID actually has the stated number of channels, bit depth, sample rate, and sample count. Without this attestation,

there would be no on-chain guarantee that the metadata accurately describes the stored audio data.

This pattern enables pluggable ingestion: different ingester modules can implement different validation strategies while the core protocol guarantees that *some* ingester attested the audio before it was created. For example, an ingester powered by Nautilus⁵ (a framework for verified computation in trusted execution environments) can download the audio file from Walrus, recompute the blob ID from the file data, extract the audio properties directly from the file's stream metadata, and sign a cryptographic attestation binding the blob ID to the verified parameters. The ingester's witness type acts as a stamp of approval on the resulting `Audio` struct, and higher-level consumers of MusicOS (applications, platforms, and agents) can make trust decisions based on which ingester attested a given audio file.

3.6 Deterministic Object Addressing

MusicOS uses Sui's derived object mechanism to generate deterministic addresses for admin capabilities and recordings. A `CompositionAdminCap` is derived from its parent composition's UID, meaning the capability's address can be computed client-side from the composition's address alone, with no indexer query required.

Similarly, a recording's address is derived from its parent composition's UID using the master audio's ingester type as a derivation key. This creates a discoverable, deterministic mapping from compositions to their recordings.

Releases use a content-addressed derivation: the release UID is derived from a Blake2b-256 hash of its recording IDs, track split values, and the creation epoch. This ensures that the same set of recordings with the same splits produces a deterministic release address.

4 The Extension System

4.1 Motivation: The WordPress Model

MusicOS is designed like WordPress: an open-source, self-hosted protocol where you install whatever plugins you want. The admin capability holder is the site admin:

⁵Nautilus. *Trustless Off-Chain Computation for Sui*. <https://www.sui.io/nautilus>.

they choose which extensions to register and accept the trust implications of that choice.

This design reflects a deliberate philosophy: the core protocol should provide the data model and lifecycle management, while all application-specific behavior lives in extensions. Revenue distribution, reward pools, licensing automation, metadata enrichment, discovery algorithms, and social features are all extension concerns. The core protocol provides the hooks; the ecosystem provides the functionality.

A more managed layer, analogous to managed WordPress hosting providers, can be built on top of MusicOS. This layer would handle custody of admin capabilities and enforce extension whitelists, providing guardrails for participants who want them without restricting the protocol's permissionless nature.

4.2 Registration and Witness-Gated Access

Extensions are registered on a per-object basis. The object owner (holder of the admin capability) calls `register_extension` with three arguments: the admin capability (proving ownership), an extension witness (identifying the extension module), and a configuration value (extension-specific settings).

```
composition.register_extension(
    &composition_admin_cap,
    my_extension::Extension(),
    config,
);
```

Once registered, an extension can access the parent object's UID mutably through `uid_mut_with_extension`, which performs two checks: that the caller provides the extension's witness type (only the defining module can construct it) and that the extension is registered on this specific object.

```
public fun uid_mut_with_extension<E: drop>(
    self: &mut Composition<S>,
    _extension: E,
): &mut UID {
    extension::assert_registered<E>(&self.id);
    &mut self.id
}
```

4.3 Raw UID Access and Future Compatibility

A critical design decision in MusicOS is that extensions receive raw `&mut UID` access to the parent object rather than a restricted API surface. This is deliberate: Sui's core primitives (fund accumulators, derived objects, coin receiving, dynamic fields) all operate on `&mut UID`. New primitives are added to the Sui framework regularly. If MusicOS wrapped each UID operation into a delegated function on the core module, every new Sui primitive would require a core protocol upgrade before extensions could use it.

Raw UID access ensures extensions can adopt new Sui capabilities immediately, without waiting for or depending on the core package. The protocol evolves with the blockchain rather than lagging behind it.

4.4 Extension Isolation and Trust Model

The trade-off of raw UID access is that a registered extension has broad access to the parent's UID and could theoretically interfere with other extensions' dynamic fields. This is an accepted consequence of the permissionless model.

Extension isolation is achieved through convention rather than enforcement:

- Each extension's configuration is stored using a phantom-parameterized dynamic field key: `Extension<phantom E: drop>()`. The phantom type `E` ensures each extension has a unique storage slot.
- Extensions can be registered and unregistered regardless of the object's lifecycle state. This is intentional; extensions like reward pools are designed to operate on published (shared) objects.
- The `register` and `unregister` functions are `public(package)`, ensuring only the MusicOS core package can add or remove extension keys through the gated entry points on each core object.

The trust model is explicit: the admin capability holder is responsible for evaluating which extensions to register. This mirrors the real-world responsibility of a WordPress administrator choosing which plugins to install.

5 Revenue Distribution and Token Economics

5.1 Share Tokens

Each composition and recording mints its own fungible share token at creation time. These tokens represent ownership stakes that determine how revenue is distributed.

Parameters:

- **Supply:** 10,000,000.000000 tokens (fixed, immutable after initialization)
- **Decimals:** 6 (enabling fractional ownership to one-millionth precision)
- **Symbol:** SHARE

The share token is initialized through a validation pipeline that enforces currency configuration correctness: the `Currency` object must have exactly 6 decimals, the symbol must be “SHARE”, the metadata capability must have been deleted (preventing later modification), and the supply must be zero prior to minting. After minting the full supply, the treasury capability is consumed to make the supply permanently fixed.

Share tokens are standard Sui fungible tokens. They can be transferred, split, merged, held in custody, or governed by any on-chain logic, including extension-defined logic such as vesting schedules, lockups, or automated market makers.

5.2 Basis-Point Revenue Splits

All revenue percentages in MusicOS are expressed in basis points (BPS), where 1 BPS = 0.01% and 10,000 BPS = 100%. This convention is standard in financial services and provides sufficient precision for music industry revenue splits without introducing floating-point arithmetic.

Two types of splits govern revenue flow:

Composition split (`split_bps`). Set on the composition, this determines what percentage of a track’s revenue is allocated to the composition side vs. the recording side. For example, a split of 5,000 BPS means 50% of track revenue flows to composition share holders and 50% to recording share holders. This split is captured on the recording at creation time, ensuring it cannot be retroactively changed by the composition owner.

Track split (`split_bps` on Track). Set when a track is added to a release, this determines what percentage of the release’s total revenue is allocated to each track. All track splits on a release must sum to exactly 10,000 BPS (100%), enforced at release creation time.

5.3 Release Revenue Distribution

When revenue is distributed on a release, the protocol executes a deterministic two-level split:

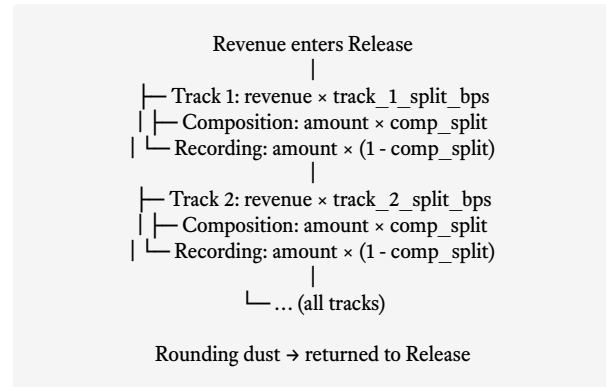


Figure 1: Two-level revenue distribution flow.

A key enabler of this flow is the *object nature* of compositions, recordings, and releases. Because each is a first-class Sui object with its own address, the release can transfer funds directly to a composition or recording’s fund accumulator via `send_funds()`, with no intermediate routing, custodial accounts, or off-chain settlement. The composition and recording objects *receive* revenue the same way any Sui address receives assets: directly and verifiably.

Once funds arrive at a composition or recording, they become available for share-token-based claiming through reward pool extensions. The entire flow, from release-level revenue to per-share-holder claims, is executed on-chain, deterministic, and independently verifiable.

Revenue distribution events, reward pool deposits, and share-token claims are all emitted using Sui’s `emit_authenticated` primitive.⁶ Unlike standard events, authenticated events can only be emitted by the defining Move package, providing cryptographic proof of origin. A light client or off-chain indexer observing a `ReleaseRevenueDistributedEvent` can therefore verify that it was definitively emitted by the MusicOS release module, not by an impersonating contract. This

⁶Sui Foundation. *Authenticated Events*. Sui Documentation, 2025.

property is critical for building trustworthy financial reporting and audit infrastructure on top of the protocol.

5.4 Reward Pools

The composition and recording reward pool extensions enable proportional revenue distribution to share token holders:

1. The object owner registers the reward pool extension.
2. Anyone can create a currency-specific reward pool attached to the object.
3. Revenue accumulated in the object’s fund accumulator is redeemed and deposited into the reward pool.
4. Share token holders stake their tokens and claim rewards proportional to their stake.

The reward pools use an *open distribution* model: any share holder can stake and claim without additional authorization. This ensures that revenue distribution is permissionless once the extension is registered.

Example. A composition has 10,000,000 SHARE tokens distributed among three songwriters: Alice (5,000,000), Bob (3,000,000), and Carol (2,000,000). When 1,000 SUI in revenue is deposited into the composition’s reward pool, Alice can claim 500 SUI, Bob can claim 300 SUI, and Carol can claim 200 SUI, proportional to their stakes.

5.5 Multi-Currency Support

Each currency type receives its own independent reward pool. A single composition or recording can simultaneously accumulate and distribute revenue in SUI, USDC, or any other Sui-native token. This is implemented through generic type parameters on the reward pool functions, requiring no protocol changes to support new currencies.

6 The Deal Protocol

The connection between recordings and releases is mediated by the **Deal** object, an explicit, on-chain authorization from a recording owner to include their recording on a specific release.

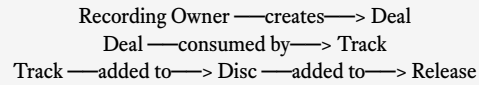


Figure 2: Deal-to-release authorization flow.

A deal captures the recording’s metadata at creation time (composition ID, revenue splits, audio ingester types, duration, cover art) and binds it to a target release ID and a track-level revenue split. The recording owner creates the deal using their `RecordingAdminCap`, providing explicit authorization for the recording’s inclusion.

When a track is created, it consumes the deal (the deal object is destroyed), transferring the authorization and metadata into the track structure. Tracks are then organized into discs and assembled into a release. This flow models the real-world process of licensing a recording for inclusion on a release: the master rights holder (recording owner) explicitly authorizes the use on specific terms.

Deals can be destroyed without being consumed if negotiations fall through, emitting a `DealDestroyedEvent` for off-chain tracking.

7 Decentralized Media Storage

Audio files, lyrics, charts, scores, and cover art are not stored on-chain. Instead, MusicOS stores references to data on **Walrus**, a decentralized blob storage network built on Sui.⁷ Each media reference is a `WalrusData` struct containing a Walrus blob ID that can be used to retrieve the data from the Walrus network.

This separation of concerns (on-chain metadata and economic logic, off-chain media storage) reflects the different requirements of each data type:

- **Metadata and economics** require strong consistency, deterministic execution, and permanent immutability. These properties are provided by the Sui blockchain.
- **Media files** require high-bandwidth storage and retrieval at low cost. These properties are provided by Walrus’s erasure-coded storage network, which achieves a 4–5× replication factor while tolerating up to two-thirds of storage nodes being unavailable.

The combination ensures that a published MusicOS object is a complete, self-contained representation of a musical work: the on-chain object contains all the meta-

⁷G. Danezis *et al.* “Walrus: An Efficient Decentralized Storage Network.” *arXiv:2505.05370*, 2025.

data and economic logic, while the Walrus references provide access to the actual audio and visual content.

8 Agent Compatibility and Programmable Music

8.1 From DDEX Messaging to Declarative Execution

The music industry’s closest existing analog to a distribution protocol is DDEX (Digital Data Exchange),⁸ a set of XML-based messaging standards used by labels, distributors, and DSPs to communicate metadata and delivery instructions. DDEX defines the *messages* (what information is exchanged) but not the *execution* (what happens when a message is received, how it is validated, or what downstream processes it triggers). Each participant in a DDEX exchange implements their own interpretation of the messages, resulting in inconsistent behavior across the industry.

MusicOS inverts this model. Rather than defining messages that participants interpret independently, MusicOS defines *state transitions* that execute deterministically on a shared runtime. Creating a composition, adding a credit, publishing a recording, distributing revenue: these are not messages to be interpreted but transactions to be executed. The Sui blockchain guarantees that every participant observes the same state and the same execution semantics.

This shift from messaging to declarative execution has profound implications for automation.

8.2 Agent-Native Interfaces

AI agents require deterministic, programmatically accessible interfaces. They cannot navigate ambiguous messaging protocols or interpret inconsistently implemented workflows. MusicOS provides exactly what agents need:

- **Deterministic state transitions.** Every function has explicit preconditions (state guards, authorization checks, constraint validations) and postconditions (state changes, event emissions). An agent can reason about what a function will do before calling it.
- **On-chain verifiability.** Every operation produces a verifiable on-chain state change. An agent does not

need to trust an API response; it can verify the state directly.

- **Composable transactions.** Sui’s programmable transaction blocks allow agents to compose multi-step workflows (create a deal, create a track, assemble a disc, publish a release) into a single atomic transaction.
- **Event-driven coordination.** Every significant state change emits a typed event (e.g., `CompositionPublishedEvent`, `ReleaseRevenueDistributedEvent`), enabling agents to subscribe to and react to protocol activity.

An AI agent managing an artist’s catalog could autonomously: monitor revenue accumulation, trigger distributions, negotiate deal terms by creating and destroying deal objects, and publish new releases, all through direct interaction with MusicOS smart contracts, or through an API layer built on top of the protocol for a more ergonomic experience.

8.3 Programmable Music as a New Distribution Format

The convergence of these properties (a faithful data model, deterministic economics, decentralized storage, and agent compatibility) produces something that does not exist in the current music industry: a *programmable* distribution format.

In the traditional industry, distributing music means delivering an audio file and a metadata sidecar (typically DDEX XML) to a DSP. The audio is a commodity. The metadata is loosely structured and inconsistently interpreted. The economics are opaque and intermediated.

In MusicOS, distributing music means publishing an on-chain object that *is* the authoritative record of the work: who created it, who owns it, how revenue flows, what audio and visual content is associated with it, and what extensions govern its behavior. The object is machine-readable, independently verifiable, and economically self-executing: a programmable entity that embodies the full creative, economic, and legal context of a musical work.

This is particularly important in an era of AI-generated content. When the cost of producing audio approaches zero, the scarce resource is not the audio itself but the human creative and economic context surrounding it. Programmable music captures that con-

⁸DDEX. *Digital Data Exchange Standards*. <https://ddex.net>.

	MusicOS	Audius	Sound / Royal	DDEX
Type	Protocol	Platform	Platform	Messaging standard
Data model	Composition / Recording / Release	Tracks	Tracks (NFT)	XML schemas
Rights structure	Full hierarchy with BPS splits	Simplified	Simplified	Comprehensive
Extensibility	Permissionless	Platform-controlled	Platform-controlled	N/A
Revenue	On-chain, deterministic	Platform-mediated	Secondary market	Per-party implementation
Storage	Walrus (decentralized)	IPFS / custom nodes	Arweave / centralized	N/A
Agent compatibility	Native	API-dependent	API-dependent	Varies
Blockchain	Sui (Move)	Solana / Ethereum	Ethereum / L2s	N/A

Table 1: Comparison of MusicOS with existing music industry technologies.

text on-chain, making it inseparable from the work it describes.

9 Comparison with Related Work

MusicOS occupies a unique position in the landscape of music industry technology:

Of particular note is Audius’s Open Audio Protocol⁹, which aims to create a “global music database” by wrapping DDEX messaging standards (ERN, MEAD, PIE) in protobuf-serialized transactions broadcast across a validator network. The approach is to conform the on-chain protocol to an existing messaging standard, placing DDEX schemas at the center of the design. This carries a fundamental constraint: the protocol inherits the limitations of a standard designed for inter-organizational messaging, not for programmable economic execution. DDEX schemas describe music metadata comprehensively, but they are not executable state. Revenue distribution, ownership enforcement, and rights-based access control are not on-chain operations derived from the stored data; they remain external concerns that the database cannot address. Audius’s Solana-based Payment Router program¹⁰ illustrates this gap: it accepts a list of recipient addresses and amounts as caller-provided

parameters and distributes tokens accordingly, a generic splitter with no connection to the music metadata stored in the Global Music Database. The caller must compute splits off-chain and pass them in.

Open Audio’s Mediorum storage layer also conflates two distinct concerns: storing audio files and delivering them to listeners. Storage of audio data is not domain-specific (audio files are bytes like any other), while low-latency delivery is a CDN concern best handled by purpose-built infrastructure. Incentivizing audio-specific storage nodes with the \$AUDIO token ties the protocol’s storage reliability to the economics of a single token. MusicOS delegates storage to Walrus, a general-purpose decentralized storage network with its own economic security model, and treats delivery as a separate concern that specialized infrastructure can address independently.

MusicOS takes a different approach to both data modeling and legacy interoperability. Rather than conforming a protocol to an outdated messaging standard, MusicOS builds a genuinely new on-chain data model (typed objects with programmable economic logic, enforceable lifecycles, and deterministic revenue execution) and bridges to legacy industry infrastructure through off-chain middleware. A Nautilus-powered API can translate between on-chain MusicOS objects and DDEX message

⁹Open Audio Protocol. *Global Music Database*. <https://docs.openaudio.org>.

¹⁰AudiusProject. *Payment Router*. <https://github.com/AudiusProject/apps/tree/main/solana-programs/payment-router>.

formats, generating on-chain attestations that bridge the protocol to traditional industry participants (DSPs, PROs, distributors) through familiar standards. This preserves the protocol’s expressiveness while providing backward compatibility, rather than sacrificing expressiveness for backward compatibility from the start.

10 Discussion

10.1 Trust Assumptions

MusicOS inherits the security guarantees of the Sui blockchain: safety and liveness under the assumption that at most one-third of validators (by stake) are Byzantine. Within this assumption, all MusicOS state transitions are deterministic and verifiable.

The extension system introduces an additional trust assumption: the admin capability holder must evaluate and accept the trust implications of each extension they register. A malicious extension with raw UID access could interfere with other extensions’ dynamic fields on the same object. This trust model is explicit and mirrors the real-world responsibility of a system administrator managing plugin installations.

The primary role of an ingester is to attest to the relationship between a Walrus blob ID and the audio file’s technical properties (channel count, bit depth, sample rate, sample count). The protocol records which ingester made this attestation but does not prescribe how the ingester verifies it. Because MusicOS is permissionless, anyone can deploy an ingester, and the protocol makes no judgment about ingester quality. Instead, the ingester’s identity (type name) is permanently recorded, enabling relying parties to evaluate the trustworthiness of the attestation and make their own trust decisions.

10.2 Protocol Neutrality

MusicOS is deployed as an immutable package with no admin-level functions, no upgrade authority, and no token-based governance. There is no protocol token, no fee extraction, and no privileged role that can alter the protocol’s behavior after deployment. This is a deliberate design choice: MusicOS is intended to be a neutral provider of programmable music primitives, free from the commercial conflicts of interest that have historically eroded trust between the music industry’s participants.

For an industry accustomed to platforms that change terms unilaterally, extract increasing fees, and leverage data asymmetries against their own users, neutrality is a prerequisite for adoption. By forgoing governance mechanisms and upgrade paths, MusicOS offers a guarantee that no future decision by any party can alter the protocol’s rules. The protocol is a public good: anyone can build on it, no one controls it, and its rules apply equally to every participant. This matters especially for institutional adoption: major labels, publishers, and collecting societies are unlikely to commit resources to infrastructure where a single startup holds governance authority over the rules of engagement. An immutable, ownerless protocol eliminates that friction entirely.

10.3 Why Sui

MusicOS’s core properties depend on Sui’s object model. Understanding this dependency clarifies why an equivalent protocol cannot be built on other major blockchains.

On EVM chains (Ethereum, Polygon, Arbitrum) and Solana, digital assets follow NFT standards (ERC-721, ERC-1155, Metaplex) that store a token ID on-chain while the metadata that defines the asset (title, credits, audio references, ownership splits) resides off-chain, typically on IPFS or a centralized API. Smart contracts operating on these assets have access to the token ID and its owner, but not to the asset’s properties. A revenue distribution contract cannot read a recording’s credit list, a licensing contract cannot inspect a composition’s split, and a marketplace contract cannot verify a release’s tracklist without first consulting an oracle or off-chain indexer. This architecture treats metadata as a display concern rather than as programmable state, which is a fatal limitation for economic protocols that must reason about asset structure to execute correctly.

Sui’s object model eliminates this constraint. Every MusicOS object (`Party`, `Composition`, `Recording`, `Release`) is a typed Move struct whose fields are fully on-chain and directly accessible to smart contracts. When `distribute_revenue` iterates a release’s tracks, it reads each track’s BPS split, composition ID, and recording ID from on-chain state in the same transaction. When a deal is created, the protocol reads the recording’s master audio ingester type, the composition’s share type, and the track’s cover art, all from on-chain fields, with no external data dependencies. Full on-chain state is a prerequisite, not a convenience: revenue distri-

Scenario	Gas Units	SUI	USD
Typical composition (3 credits, demo)	~3M	~0.0017	~\$0.002
Typical recording (8 credits, 5 stems)	~5M	~0.0028	~\$0.003
Typical single (1 track, 2 credits)	~3M	~0.0017	~\$0.002
Max recording (150 credits, 50 stems, 70 artists)	~385M	~0.21	~\$0.21
Max release (255 tracks, 20 discs, 50 credits)	~23M	~0.013	~\$0.01

Table 2: Gas cost estimates at mainnet gas price (550 MIST/unit, 1 SUI = \$1 USD).

bution that depends on off-chain metadata cannot be deterministic, and ownership structures that smart contracts cannot read cannot be enforced programmatically.

Equally important is Sui’s support for object lifecycles. MusicOS objects are not static records; they progress through defined states. A `Composition` moves from draft to published. A `Track` transitions from `Unassigned` to `Assigned`. A `Deal` is a hot potato that must be consumed in the same transaction it is created. These lifecycle transitions are enforced by the Move type system and the Sui runtime, not by application-level checks that could be bypassed. On EVM or Solana, lifecycle enforcement requires manual state-machine patterns with no compiler guarantees, and the absence of linear types means that hot-potato flows (where an object must be used exactly once before the transaction completes) are not expressible.

On-chain data and on-chain lifecycles together make economic protocols practical. MusicOS requires both: the data to compute revenue splits, and the lifecycles to ensure that deals are honored, tracks are assigned exactly once, and compositions cannot be modified after publication. Sui is the only production blockchain where both properties hold natively.

10.4 On-Chain Cost Analysis

A concern with any on-chain protocol is whether the cost of creating rich, fully-populated objects is prohibitive. MusicOS provides generous limits (up to 150 credits per recording (each with up to 10 roles), 100 stems, 20 primary artists, 50 featured artists, 255 tracks per release across 20 discs), all well within Sui’s 250 KB maximum object size. Gas profiling of kitchen-sink tests that exercise these maximum bounds confirms that costs are negligible:

A typical recording with real-world parameters (a handful of credited contributors, a few stems, one or two primary artists) costs under half a cent. Even the most extreme case, a maximally-populated recording object that no real-world production would approach, costs about 21 cents. On-chain metadata size is not a practical constraint.

For perspective, consider what it would cost to approximate the same feature set through traditional legal infrastructure. Establishing a comparable ownership and revenue structure for a single recording (drafting and executing split agreements among multiple rights holders, registering copyrights, setting up escrow or collective administration for royalty distribution, and filing the necessary paperwork with PROs, the MLC, and distributors) routinely costs thousands to tens of thousands of dollars in legal fees, administrative overhead, and organizational dues. Transferring ownership shares requires new agreements, often with legal counsel on both sides. Auditing revenue flows requires forensic accountants. On MusicOS, all of these operations (creating an immutable ownership record, encoding revenue splits, transferring share tokens, distributing revenue deterministically) execute for fractions of a cent, with full auditability built in at no additional cost.

10.5 Limitations and Trade-offs

Composition split immutability on recordings. A recording captures the composition’s `split_bps` at creation time. If the composition owner later wishes to change the split (before publishing the composition), existing recordings retain the old split. This is intentional (it prevents retroactive changes to negotiated terms), but it requires that splits be finalized before recordings are created.

No on-chain dispute resolution. MusicOS does not include mechanisms for resolving ownership disputes,

takedown requests, or copyright claims. These are inherently off-chain processes that involve legal jurisdictions and human judgment. MusicOS provides the verifiable data substrate that such processes can reference, but it does not replace them.

Extension interference. The raw UID access model accepts the possibility of extension interference in exchange for future compatibility. Managed layers built on top of MusicOS can mitigate this through extension whitelisting and auditing.

11 Conclusion

The music industry’s infrastructure was built for an era of physical distribution and centralized intermediation. It has not meaningfully evolved to serve an era of digital abundance, AI-generated content, and programmable economic relationships. The consequences are measurable: billions in unmatched royalties, opaque revenue pipelines, and a distribution format (the audio file) that captures none of the creative, economic, or social context that gives music its value.

MusicOS provides an alternative foundation. By encoding the full structure of music rights into a permissionless, extensible on-chain protocol, MusicOS makes the ownership, attribution, and economics of music verifiable, deterministic, and programmable. It does not seek to replace the music industry’s participants but to give them infrastructure worthy of the creative work they support.

The protocol will be deployed in Q2 2026 and is available under the Apache 2.0 license.

References

- [1] IFPI. *Global Music Report 2024*. International Federation of the Phonographic Industry, 2024.
- [2] U.S. Copyright Office. *Mechanical Licensing Collective: Annual Report*. Library of Congress, 2023.
- [3] G. Danezis, G. Giuliani, E. Kokoris Kogias, M. Legner, J.-P. Smith, A. Sonnino, and K. Wüst. “Walrus: An Efficient Decentralized Storage Network.” *arXiv:2505.05370*, 2025.
- [4] Sui Foundation. *The Move Programming Language*. 2024.
- [5] DDEX. *Digital Data Exchange Standards*. <https://ddex.net>.
- [6] Nautilus. *Trustless Off-Chain Computation for Sui*. <https://www.sui.io/nautilus>.

[7] Sui Foundation. *Authenticated Events*. Sui Documentation, 2025.

[8] SuiNS. *Sui Name Service*. <https://suins.io>.

MusicOS is open-source software licensed under Apache 2.0. The protocol is developed by Unconfirmed Labs.